

Portabelt informationssystem för turister



Författare:

Johan Lindberg, iq@acc.umu.se
Markus Lindblom, di00mlm@ing.umu.se

Innehållsförteckning

INLEDNING.....	3
METOD.....	4
FUNKTIONSBESKRIVNING	5
HÅRDVARA.....	6
TUXXEN	6
GPRS-MODULEN	6
GPS-MODULEN.....	6
STRÖMFÖRSÖRJNING OCH GPRS-KORTET	7
PROGRAMBESKRIVNING.....	8
TUXXEN	9
SERVERN	9
ÖVRIGA PROGRAM.....	9
ANVÄNDARBESKRIVNING.....	11
RESULTAT	12
SAMMANFATTNING.....	13
BILAGOR.....	14
KRETSSCHEMAN	14
<i>Strömförsörjning</i>	14
<i>GPRS</i>	15
<i>Serieinterface</i>	16
KOMPLEMENTLISTA.....	17
KÄLLKOD	18
<i>server.c</i>	18
<i>main.c</i>	25

Inledning

Användningen av portabla informationssystem kommer att öka i vårt samhälle. Informationsmängden ökar och dom tillgängliga teknikerna för att komma åt informationen ökar.

Vad är ett portabelt informationssystem då? Ja i vårt fall så rör det sig om ett system gjort för turister i syftet att komma åt information om sevärdheter och sånt som kan vara av intresse för en turist, t.ex. affärer, matställen, tågstationer, flygplatser o.s.v..

När man bygger ett system som detta måste man ha i åtanke vilka som kan tänkas använda det. Vi måste ha i åtanke att personen som använder systemet för tillfället inte kan något om tekniska prylar. Därför måste man se till att gränssnittet blir så enkelt som möjligt och inte möjliggör att man kan göra nått för att antingen förstöra eller tappa bort sig i menyerna/systemet.

Valet av en Philips Screenphone (Tuxscreen som den nu kallas efter uppköpet) gjordes för att den har både färgskärm och touchscreen, samt att den har en StrongARM processor med MMU vilket möjliggör användandet av Linux. Eftersom man kan använda sig av Linux så får man också mycket gratis i form av drivrutiner och applikationer som kan tänkas behövas för informationssystemet.

Metod

Efter att vi hade hämtat ut vår Tuxscreen så började vi genast att undersöka och testa den för att se hur den fungerade. Detta började vi direkt med första veckan.

Vi insåg snabbt att vi inte kunde göra mycket med den före vi hade bytt ut det befintliga operativsystemet Inferno som fanns på den. För att kunna byta ut det befintliga operativsystemet så måste man först låsa upp flashminnet som sitter i den. Efter att flashminnet var uppläst kunde den befintliga bootloadern bytas ut mot BLOB, vi kunde då ladda ner en imagefil som bestod av filsystem och Linuxkärna.

För att kunna korskompilera applikationer och annat så måste man ha en korskompilator för ARMprocessorn. Vi valde att ladda ner den så kallade "buildrooten" för Tuxxen som innehöll filsystem, utvecklingsverktyg m.m..

Buildrooten laddades ner från CVS. När man sedan gjorde en make så laddar den automatiskt ner dom källfiler som behövs och bygger korskompileringsverktyg m.m.. Efter att den hade byggt allt klart så gör den en imagefil som innehöll kärna och ett filsystem med några program. Vi valde även att från början kompilera in stöd för MicroWindows för att på så sätt kunna göra ett enkelt och intuitivt användargränssnitt.

När Linux väl var installerat på Tuxxen så kunde vi börja se hur vi skulle koppla vår hårdvara till den. För att kunna lösa uppgiften med att hämta information från en databas och veta vars informationsenheten var så använde vi oss av GPRS och GPS, där GPRS-modulen användes för att koppla upp oss mot Internet och sedan kontakta en databas server som kör MySQL. GPS-modulen används för att kolla positionen. Detta enda som skickas till databasen är just denna position. Som ett krav i kursen så ingick det även att alla projekt skulle kunna hitta varann med Bluetooth, för detta så blev det ett PCMCIA blåtands kort som vi använde oss av, dels för att Ericsson-modulen fick vi aldrig att funka med OpenBT stacken eller Bluez stacken och dels för att vi inte hade tid att göra en egen stack för att få den att funka (vilket hade funkade då vi fick kontakt med modulen om vi skickade opkoder direkt till modulen via en egen applikation). PCMCIA modulen funkade utan några större problem så vi valde att ta den istället för att kunna vara säkra på att hinna klart till kursens slut.

När vi hade valt vår hårdvara så undersökte vi vad som behövdes för att kunna använda denna hårdvara. GPRS-modulen t.ex. behövde vi tillverka ett kretskort åt och eftersom vi var först med att tillverka detta kort så blev vår lösning inte direkt någon höjdare. Vi hittade några fel på kortet och upptäckte även att Tx och Rx från GPRS-modulen var tvärtom vad det ska vara. Anledningen till detta har vi fortfarande inte någon vettig förklaring till.

Vi valde att lägga strömförsörjningen på samma kort som GPRS-modulen så behövde vi inte göra två separata kort. Vi kom fram till att vi skulle behöva tre olika spänningar ifall vi skulle kunna driva både Tuxxen, GPRS-modulen och GPS-modulen. Blåtanden drivs via PCMCIA slotten så den behövde vi inte oroa oss över.

Vi inriktade oss på att få igång GPRS-modulen först eftersom det var något som vi aldrig hade hållit på med förut. Detta tog den mesta av tiden på kursen dels eftersom vi fick lov att göra kretskortet till den samt att vi fick läsa på hur det skulle funka. Då vi väl kunde testa GPRS-modulen gick det rätt så fort att upprätta en anslutning.

GPS-modulen var inte några större problem med så den hade ganska låg prioritet. Blåtanden strulade rätt så mycket innan vi bestämde oss för att byta till PCMCIA versionen.

Funktionsbeskrivning

Den primära hårdvaran i vårt projekt är en Tuxscreen. GPRS-modulen är kopplad via ett seriellt interface mot Tuxxen och har hastigheten 9600 baud, detta är serieport `/dev/ttySA0` på Tuxxen. GPS-modulen är kopplad till `/dev/ttySA1` och arbetar på 4800 baud.

Till GPRS-modulen använder man sig av AT-kommandon för att göra inställningar på den. Efter att en datauppkoppling är gjord så kommer GPRS-modulen att gå över i dataöverföringsläge och tar då endast emot Escapesequenser i form av `+++`. För att lägga på då så skickar man `+++ATH` t.ex..

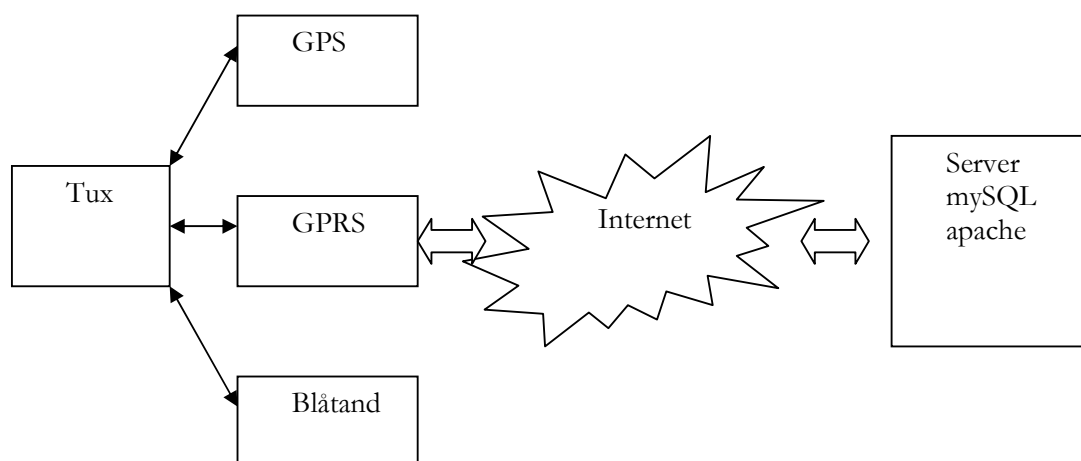
GPS-modulen skickar kontinuerligt strängar med GPS information. Det man gör då är att läsa av den strängen som innehåller positions information, den börjar med `GPGGA`.

Som nämnt tidigare så körs MicroWindows som fönsterhanterare och vi har då skrivit ett användargränssnitt där som sköter all kommunikation med användaren. Vi använder även TinyWidgets som är ett bibliotek till nano-X för att enkelt kunna använda knappar och listor och dylika saker. Ifall användaren gör en sökning på det område som enheten är på för tillfället så kommer först GPS-modulens position att läsas av. Därefter så skickas positionen med GPRS över till en server som kör en MySQLdatabas innehållandes information om platser på olika koordinater. Det som då görs är att ett valbart område runt den position som skickades kommer att kollas. Servern skickar tillbaka information om vilka platser som finns kring den positionen som skickades från början.

I användargränssnittet så får man då upp vilka platser som finns i närheten. Man kan då välja någon av dom platserna och få upp specifik information om just den platsen som man har valt.

När det gäller blåtanden så är PCMCIA kortet startat samtidigt som Tuxxen startade. Detta görs genom att anropa programmet `bti` som startar OpenBT Bluetooth stacken. Efter att detta är gjort så görs sökningar med programmet `btinc`.

Ifall användaren gör en sökning av omkringliggande blåtandsenheter så kommer programmet `btinc` att anropas och dom funna enheterna kommer att skrivas ner i en textfil som sedan parsas. Filen `bt_if.c` som används vid kompilering av `bti` och `btinc` är modifierad så att den istället för att skriva ut på skärmen vilka enheter som finns så skriver den till en fil istället.



Hårdvara

Tuxxen

Grunden i projektet är Tuxxen som är en StrongARM baserad plattform med 4MB flash och 16MB RAM. Den har stöd för PCMCIA kort och har 2st serieportar som går att använda. En av dessa serieportar är normalt inte utlörd så denna måste man löda ut från IO-kortet som sitter i Tuxxen. Den har även andra serieportar men dessa används till bland annat IR-tangentbordet som finns. Tangentbordet som finns till Tuxxen är ett litet kompakt tangentbord som vi valde att behålla då det annars skulle bli svårt att ändra något ifall nått inte stämmer eftersom vi har GPRS-modulen på den första serieporten. Den serieporten har man normalt inloggning på (det är en *getty* startad på den konsolen).

GPRS-modulen

Är en Sony-Ericsson GR47 GSM/GPRS modul som har tre serieinterface. Det första interfacet är det som används för kommunikation med själva GPRS-modulen och är RS232 kompatibel, dock så använder den sig av 2.7Volts nivåer vilket gör att man kan inte ansluta den direkt mot en 3.3Volts anslutning. I vårt fall så använde vi oss av en MAX3232 krets som vi kopplade in mot Td och Rd på GPRS-modulen. I GR47:ans fall så har dom av någon outgrundlig anledning bytt Td och Rd så att inkopplingen är tvärtom vad det i normala fall ska vara. Td är alltså det som skickas FRÅN den anslutna datautrustningen, alltså det som skickas från Tuxxen till GR47:an. Annars är anslutningarna till GPRS-modulen ganska enkla och i vårt fall så har vi bara använt oss av serieportarna och några andra anslutningar för t.ex. en lysdiod. Sedan är det bara strömförsörjningen som ska kopplas in. GR47:an vill ha från 3.4Volt upp till 4Volt. Dock så märkte vi att ifall strömförsörjningen sjönk ner mot 3.6Volt så började den starta om sig själv (vilket den inte bör göra) och vi fick då problem med att den läste sig när man skrev in AT-kommandon. Höjde vi spänningen till 3.8Volt så försvann detta problem. Man skulle kunna gissa på nått fel på strömförsörjningskortet fast den spänningen som kommer ut från det kortet är inget fel på. Så vi kan bara gissa på att det är modulen som är känslig eller att vi fick in störningar från nått.

Från början var det tänkt att vi skulle koppla in GPS-modulen på GPRS-modulens andra serieport då denna är gjord för att kunna användas med en GPS. Det man gör är att säga till GPRS-modulen (med AT-kommandon) att spegla den andra serieporten till den första. Så den informationen som kommer på serieport två leds helt enkelt till serieport ett. Vi testade att göra detta och det funkade väldigt bra, dock så kan man inte använda sig av GPRS för dataöverföring samtidigt som detta görs utan då måste man koppla bort speglingen och koppla upp sin GPRS-anslutning för att sedan koppla ner den och fortsätta spegla den andra serieporten. I vårt fall hade detta inte varit något problem då vi hade först kunnat ta positionen, sedan kopplat upp anslutningen och sedan kopplat ner den igen. Det hade blivit lite svårare att få ett stabilt system, helst eftersom vi hade problem med att kunna koppla upp GPRS-anslutningen.

Nu visade det sig att vi inte behövde använda oss av speglingen för att komma åt GPS-modulen. Anledningen från början till att vi skulle göra så var för att vi inte hade tillräckligt med serieportar. Men efter att vi valde PCMCIA kortet på grund av tidsbrist så fick vi en serieport över. Denna använde vi till GPS-modulen istället.

GPS-modulen

Är en Furuno GN-79 och stödjer NMEA-protokollet. Den har ett serieinterface som stödjer 3.3Volts nivåer vilket möjliggör inkoppling direkt på Tuxxens andra serieport. Här behövde vi alltså inte använda oss av en MAX3232 krets för att kunna koppla in den.

GPS-modulen skickar ut en tre strängar varje sekund men det är bara GPGGA-strängen som vi vill åt. GPS-modulen är igång hela tiden som Tuxxen är påslagen och den skickar data på serieport /dev/ttySA1.

Strömförsörjning och GPRS-kortet

För att kunna göra Tuxxen bärbar (batteridrift) så var vi tvungen att göra ett strömförsörjningskort som gav ut 3.3Volt och 5Volt. På detta kort så satte vi även våra MAX-kretsar och GPRS-modulen. Vi var tvungna att ha GPRS-modulen på ett kretskort eftersom anslutningslisten på modulen är en stiftlist som gjorde att vi kunde inte bara ansluta strömförsörjning och serieanslutningarna direkt på själva modulen.

Eftersom vi gjorde strömförsörjning åt Tuxxen så la vi även strömförsörjningen åt GPRS-modulen på samma kort. Vi valde att implementera 3.3 och 3.6Voltsspänningarna med hjälp av justerbara switchade spänningsregulatorer. Detta visade sig vara väldigt tursamt då vi fick ställa upp både 3.3Voltsspänningen och även 3.6Volten. Tuxxen hade en egenhet att den startade om ifall 3.3volten var för låg vilket den blev när vi belastade strömförsörjningskortet. Därför ställde vi upp den spänningen.

Dom switchregulatorer som vi valde var LM2576 och klarar 3A, detta kan tyckas vara mycket men GPRS-modulen kan toppa någon ampere. GPRS-modulen har en medelström på ungefär 350mA vid dataöverföring och GN-79:an ska ligga på ungefär 79mA. Tuxxens strömförsörjning för nätet klarar över 1.5Ampere på 3.3volten och 0.6Ampere på 5volten.

Nu är vårt kort lite väl överdimensionerat både vad gäller fysik storlek på kortet och vad gäller vad det klarar. Men vi var lite stressade med att få igång GPRS-modulen eftersom vi behövde testa den och lära oss hur den fungerade därav det inte så perfekta kortet.

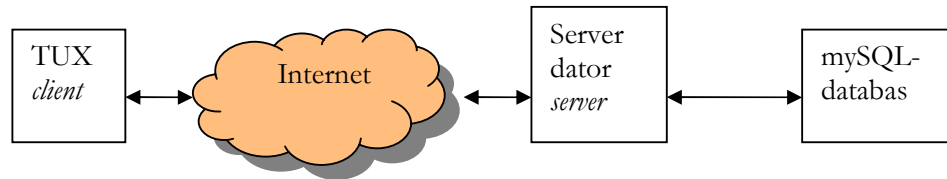
Serieportskretsarna som användes var två stycken MAX3232 som innehåller vardera två stycken in och ut kanaler. Alltså en för TTL till RS232 nivåer och en från RS232 till TTL. Det är dubbla sådana kanaler i varje krets. Utöver dom komponenterna så finns det diverse kondensatorer och spolar för strömförsörjningsbiten. Kretsschema och komponentlista följer med som bilagor.

Konstruktionen för själva strömförsörjningen är tagen direkt från applikationsbladet för LM2676-kretsarna. Likaså för MAX-kretsarna.

GPRS-modulen sitter fast i en stiftlist på kortet.

Programbeskrivning

Den viktigaste mjukvarudelen i projektet är server, klient mjukvaran som kommunicerar mot Tuxxen och mot databasen. Följande bild åskådliggör detta:



Här ser man att Tuxxen kör som *client* och innehåller klientmjukvaran som kopplar upp sig mot servern via Internet. Servern kör ett liknande program som heter *server*, detta program har till uppgift att ta emot förfrågningar från klienten och hämta data från mySQL databasen som ligger på samma dator som serverprogrammet och.

Tillvägagångssättet för en förfrågning från klienten går lite enkelt till på följande sätt:

1. Klienten skickar latitud.
2. Servern tar emot och begär nästa.
3. Klienten skickar longitud.
4. Servern tar emot och begär nästa
5. Klienten skickar storleken på området som användaren vill ha info om.
6. Servern tar emot.
7. Servern jämför positionerna med de i databasen.
8. Servern skickar de positioner som är inom området till klienten.

För att servern ska kunna veta vilka platser som finns i närheten av den position som skickades som beräknas även avståndet till närliggande punkter ut för att veta ifall dom ligger inom det avstånd som är inställd av användaren. När avståndet beräknas tas även hänsyn till jordens krökning för att man ska kunna använda systemet på lite större avstånd.

När sedan användaren får upp en lista om dom platser som finns i området så kan användaren välja att trycka på en plats och begära information om just den platsen. En förfrågan skickas då till servern om information om den platsen och servern kommer då att göra en SQL_Query om just den platsen, databasen kommer då att skicka tillbaka informationen till servern som i sin tur skickar den till klienten.

När servern alltså ska beräkna avståndet mellan punkter och avgöra vilka platser den ska skicka tillbaka så görs detta:

- `SELECT * FROM test`

Självklart är detta inte så smart om man har en större databas eftersom här skickar vi tillbaka alla poster från databasen för att kunna jämföra. Men det kan man lösa genom att dela in databasen i områden.

När sedan servern har räknat fram avståndet mellan alla platser och avgjort vilka som är inom det område vi vill ha information om så skickar den tillbaka namnet på dom platserna, användaren får upp dessa platser och väljer en plats som han/hon är intresserad av. Det som görs då i servern är följande:

- `SELECT info FROM test WHERE location=platsen_klienten_vill_ha_info_om`

Där *test* är databasen och *location* är platsen som vi vill ha information om. Databasen returnerar då informationen om just den specifika platsen och sedan skickas denna information tillbaka till klienten där en informationsruta visar det som är begärt.

Tuxxen

På Tuxxen ligger som redan känt en klient del. Detta program sköter även om den grafiska biten och använder sig av TinyWidgets som är ett grafiskt paket för små inbyggda system. X-servern som körs är nano-X och fönsterhanteraren är nanowm.

När Tuxxen startas kommer en rad saker att utföras. Blåtandsstacken är inkompilerad i kärnan och startas därför när Linux startas. Efter ett tag så kommer ett skript att köras, *start* som ligger i root katalogen. Detta skript har till uppgift att först kolla om vissa speciella devices som behövs av OpenBT finns. Ifall dom inte finns så skapas dom med *makedevices*, ett skript som helt enkelt bara gör dom devices (mknod) som behövs. Därefter så anropas programmet *bti* för att starta stacken.

Efter att stacken är startad så kommer *pppd* att anropas för att starta GPRS uppkopplingen. Den startas med *pppd call gprs*, där *gprs* är en textfil som innehåller dom svar som GPRS-modulen ska svara med efter att ett visst kommando har skickats. Programmet *pppd* använder sig av ett annat program *chat* som har till uppgift att sköta "pratet" med GPRS-modulen, *chat* skickar ett AT-kommando och väntar på ett svar, sedan går den vidare till nästa kommando och så vidare. När *pppd* har lyckats göra en anslutning mot Telia så sätter den automatiskt IP-adress m.m. på *ppp0* som är den nätverksenheten som används för kommunikation via Internet. Den enheten är satt som default route så allt som har med Internet att göra kommer att gå via den. Hade man haft ett vanligt nätverkskort t.ex. så skulle det ha varit *eth0*

Efter att dessa saker är gjorda så startas X med skripten *startx*. I den skripten har vi lagt till att vår applikation ska startas, *demo1* heter den.

Efter att nano-X har startat så startar applikationen och ett fönster med tre val öppnas. Första valet är för att hämta information om den plats man är på för tillfället, andra valet är för att lista alla blåtandsenheter i närheten och sista valet är för att ställa in hur stor radie man vill ha information om.

Ifall man trycker på det första valet så kommer först den nuvarande GPS-positionen att läsas av från GPS-modulen, detta är väldigt enkelt då man bara behöver göra en *fgets* för att läsa av en serieport. Sedan kommer klienten med hjälp av sockets att koppla upp sig mot servern och skicka den aktuella positionen. När informationen sedan kommer tillbaka så listas alla platser inom området. Man kan då välja en plats och hämta mer information om den platsen.

Trycker man på knappen för att söka efter blåtandsenheter så kommer programmet *btinq* att köras och skriva sitt resultat till *result.txt*. Applikationen läser sedan filen och listar dom enheter som hittades, filen skrivs över varje gång så man behöver inte ta bort den.

Delen för att sätta storleken på området sätter helt enkelt bara en variabel som sedan används när man skickar information till servern.

Servern

När man startar servern så anger man vilken port den ska startas på. Denna port används för att lyssna efter anslutningar från klienten på Tuxxen. Servern sköter all kommunikation med databas servern som ligger på samma dator. För att kunna lägga till information till databasen så finns några .php sidor för att skriva information till databasen. På datorn så körs Linux med Apache och MySQL samt PHP.

Övriga program

Dom programmen som vi inte har skrivit själva är tagna direkt från Internet och korskompilerade för att fungera på Tuxxen. För att det ska gå att korskompilera så måste man ha en korskompilator. Denna fick vi genom att använda oss av buildrooten som fanns för Tuxxen. En buildroot är en samling program och verktyg som används för att göra ett helt filsystem inklusive ett antal applikationer som man kan tänkas behöva. I Tuxxen:s fall så laddade vi ner buildrooten via CVS från Sourceforge. För att ladda ner via CVS så skriver man följande:

- `cvs -d:pserver:anonymous@cvs.sourceforge.net:/cvsroot/tuxscreen login`
- `cvs -z3 -d:pserver:anonymous@cvs.sourceforge.net:/cvsroot/tuxscreen co buildroot-tux`

När man sedan har laddat ner allt så kör man helt enkelt en *make* för att göra ett komplett filsystem. Make skriptet kommer att ladda hem dom filer som behövs och göra korskompilatorn samt alla verktyg och applikationer. Därefter görs en image fil i formatet *.jffs2* som man laddar ner till Tuxxen.

På samma sätt som buildrooten så laddas OpenBT stacken ner via CVS:

- `cvs -d:pserver:anonymous@cvs.sourceforge.net:/cvsroot/openbt login`
- `cvs -z3 -d:pserver:anonymous@cvs.sourceforge.net:/cvsroot/openbt co all`

Verktygen måste man dock korskompilera med hjälp av den kompilatorn som finns i buildrooten. Man skickar med parametrarna *CC=sökväg_till_gcc* *LD=sökväg_till_ld* till make genom att skriva *make CC=sökväg LD=sökväg*. Det man behöver är förutom själva drivrutinerna vissa verktyg som ligger under en katalog som heter *experimental*. Drivrutinerna läggs in i kärnans katalogstruktur under *char* katalogen och sedan väljer man att använda OpenBT stacken istället för Bluez stacken när man kompilerar kärnan. För att ändra i kärnan gör man en *make menuconfig* (alternativt *make config*) och sedan *make dep* och sist *make clean* detta för att man sedan i huvudkatalogen till buildrooten gör en *make*. Detta gäller alla saker man vill ändra på, man använder sig alltid av buildrootens make skript för att göra allting. Vi fick lov att ändra i busybox t.ex. för att få stöd för programmet *stty* och istället för att då ladda ner hela imagefilen tog vi bara busybox filen efter att allt var kompilerat och förde över den filen till Tuxxen via nollmodem. Detta gällde det mesta vi gjorde, vi gjorde först applikationen på utvecklingsdatorn och sedan överförde vi filerna till Tuxxen med nollmodem.

Användarbeskrivning

När allt funkar som det ska göra så behöver man bara slå igång enheten. Den kommer då att starta Bluetooth och även GPRS-anslutningen till Telia. För tillfället så visas all text från dom olika programmen och detta skulle man kunna ta bort.

När sedan användargränssnittet är startat så får man upp tre knappar. Den första knappen är för att söka efter platser kring den position man står vid. Den andra knappen listar alla Bluetooth enheter som är inom räckhåll och den sista knappen används för att ställa storleken på den area man vill leta efter sevärdheter inom. Efter man har valt första knappen så får man upp en lista på sevärdheter som är inom räckhåll och man kan i denna lista välja en sevärdhet och trycka på knappen Get info och då får man fram den information som är lagrad om just den platsen. Det grafiska gränssnittet är gjort med tanke på enkelhet och vi har även valt att lägga stora områden svarta så att Tuxxen skall dra lite mindre ström.

Resultat

När allt fungerar så funkar enheten precis som det var tänkt. Vi kan ta in en GPS koordinat, skicka en förfrågan till vår server och få ett svar från den om dom platser som man kan tänkas vilja kolla upp.

Ibland kan det dock vara svårt att få kontakt via GPRS-modulen, förmodligen på grund av dess antenn. Det som var beställt till alla GPRS-moduler var en jordplansantenn avsedd för bilmontering eller på ett större jordplan. Sedan så är ju GPRS inte direkt något snabbt överföringsmedium, det som är dåligast är ju responstiderna. Oftast tar det över 1 sekund när man pingar en IP-adress. Detta gör att applikationen oftast inte flyter på så bra. Mängden data som överförs är ju inte stor och hade responstiderna varit lägre så hade det flutit på utan problem.

Något som har fungerat dålig är GPS-modulerna. Oftast så får man ingen kontakt med satelliterna. Dom nyare enheterna som kopplas direkt på serieporten med PS2 strömförsörjning är dock problemfria. Dock så hade vi Furuno modellen.

För att klaga ytterligare på lite hårdvara så måste man nog nämna Ericsson Bluetooth enheten. Den vållade oss så mycket bekymmer att vi valde att använda oss av PCMCIA modellen istället då tiden var väldigt knapp. Vi kunde få kontakt med ericsson-modulen ifall vi skickade kommandon direkt via en applikation, dock så kunde vi inte lägga ner någon tid på att skriva en egen stack (även om den inte hade behövt vara så stor då vi bara skulle göra inquiries) då detta var i slutet av kursen. När vi använde oss av OpenBT som stack så lyckades *bti* programmet aldrig hitta modulen. Vi kunde ha använt oss av Bluez stacken men då kunde vi istället inte korskompilera verktygen som behövdes då vi använde oss av uClibc på Tuxxen. Dom som körde LART:en använde vanliga glibc och då var det inga problem att använda sig av Bluez. Förmodligen skulle man kunna korskompilera Bluez då det på hemsidan står att det är möjligt.

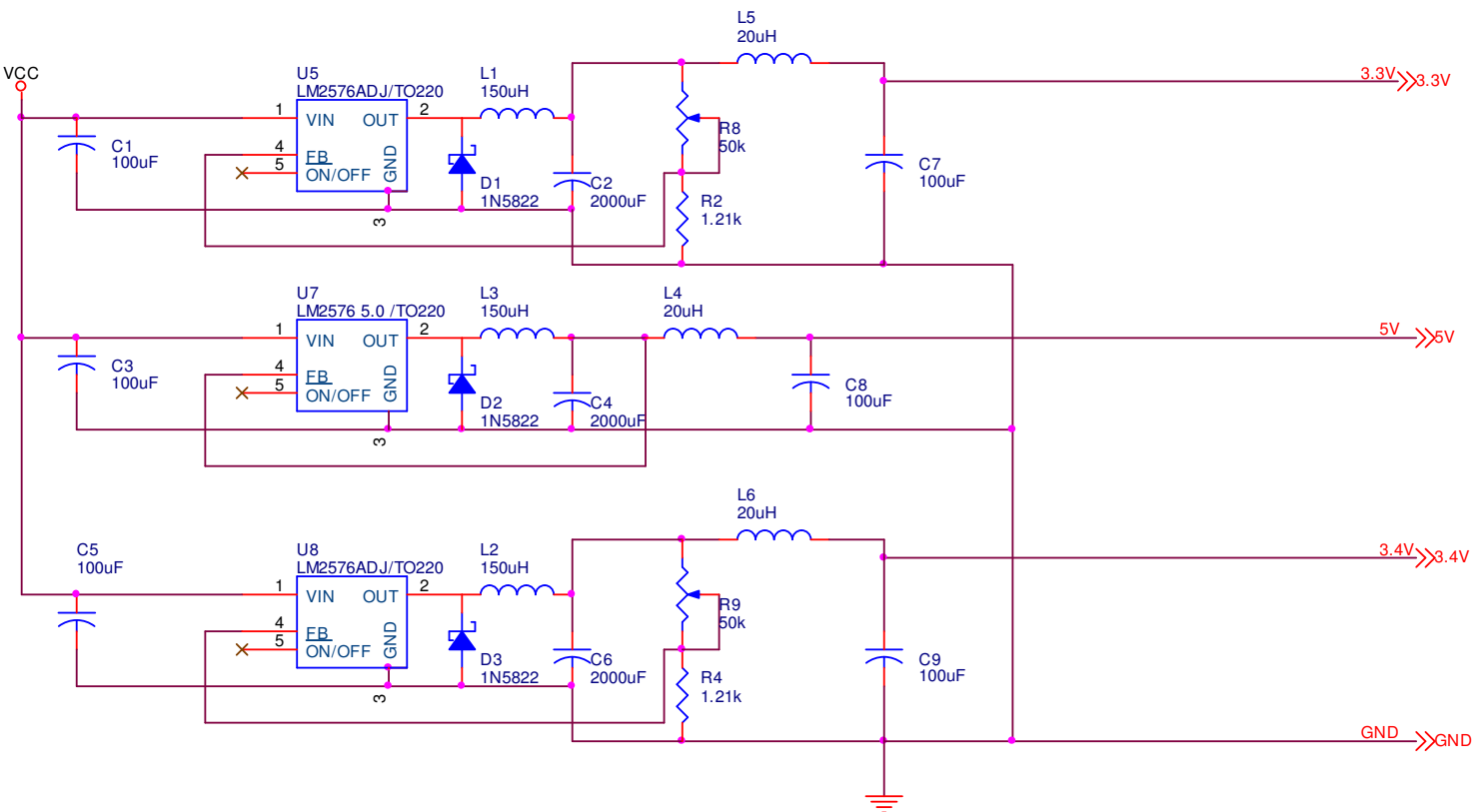
Sammanfattning

Nu i efterhand när man kollar på vilken hårdvara som vi hade valt så skulle man kunna ha valt t.ex. en Ipaq som grund istället för Tuxxen, dels för att Tuxxen är onödigt stor och klumpig och har egentligen inte något mer än vad t.ex. en Ipaq har. Skulle man valt en Ipaq så skulle man kunna göra allt mycket mindre och man skulle förmodligen välja en annan GPRS-modul också då GR47:an är mer för fasta mobila applikationer, t.ex. i ett fordon. Tar man t.ex. en nyare mobiltelefon idag så saknar dom oftast bara en GPS-modul.

Bilagor

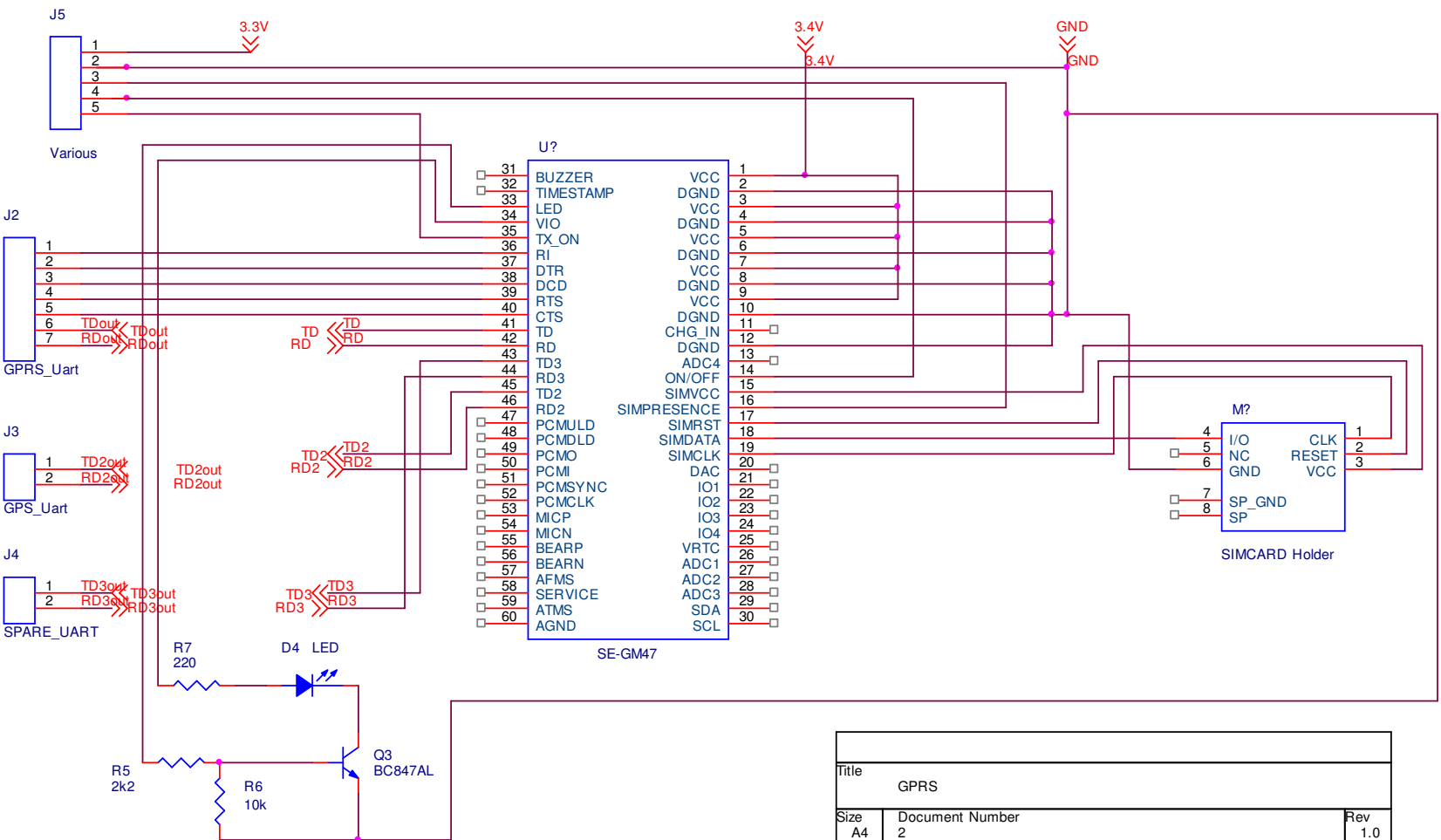
Kretsscheman

Strömförsörjning



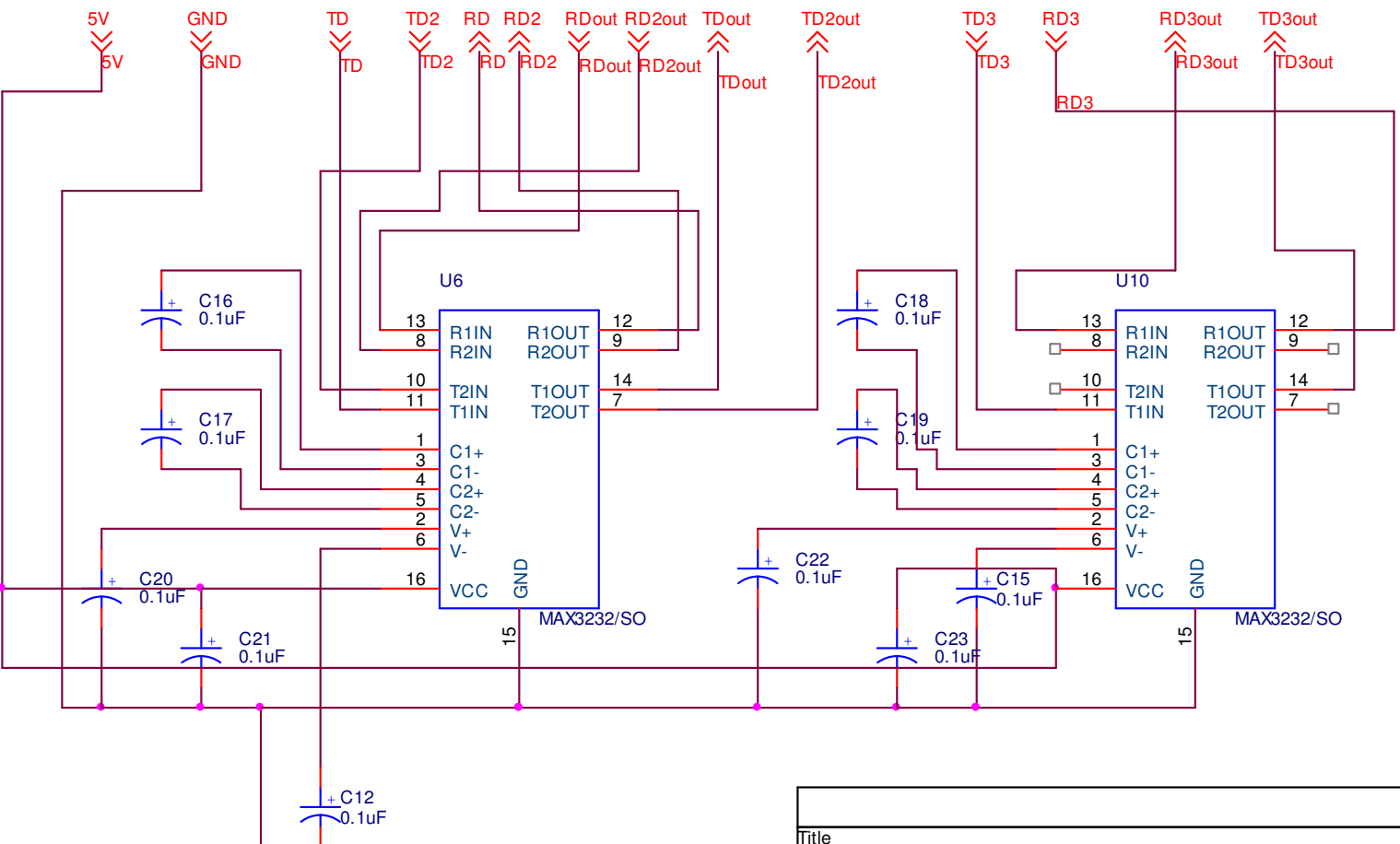
Title		
Strömförsörjning		
Size A	Document Number 1	Rev 1.0
Date:	Thursday, September 25, 2003	Sheet 1 of 1

GPRS



Title		
GPRS		
Size	Document Number	Rev
A4	2	1.0
Date:	Thursday, September 25, 2003	Sheet 1 of 1

Serieinterface



Title		
Serieinterface		
Size	Document Number	Rev
A4	3	1.0
Date:	Thursday, September 25, 2003	Sheet 1 of 1

Komponentlista

Antal	Komponent, Värde	Beskrivning	Referens
6	100uF	Kondensator	C1, C3, C5, C7, C8, C9
3	2000uF	Kondensator	C2, C4, C6
9	0.1uF	Kondensator	C12, C15-C23
1	1N5822		D1, D2, D3
1	LED		D4
1	GPRS_Uart – 7polers stiftlist		J2
1	GPS_Uart	2polers stiftlist	J3
1	SPARE_UART – 2polers stiftlist		J4
1	Various	5polers stiftlist	J5
3	150uH	Spole	L1, L2, L3
3	20uH	Spole	L4, L5, L6
1	SIMCARD Holder		M1
1	BC847AL		Q3
2	1.21k		R2, R4
1	2k2		R5
1	10k		R6
1	220ohm		R7
2	50k vridresistor		R8, R9
1	SE-GR47		U1
2	LM2576ADJ/TO220		U5, U8
2	MAX3232/SO		U6, U10
1	LM2576 5.0 /TO220		U7

Källkod

server.c

```
/*
Wearble Computers 10p
Markus Lindblom, Johan Lindberg
SERVER delen.
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/utsname.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <unistd.h>
#include <mysql/mysql.h>
#include <math.h>

#define LINUX TRUE
#define MYSQL_HOST "localhost" // database server name (localhost
= den här datorn)
#define MYSQL_DB "test" // Namnet på databasen
#define MYSQL_USERID "" // database user id
#define MYSQL_PASSWD "" // userid password

// MYSQL GLOBALS
MYSQL mysql;
MYSQL_RES *result;
MYSQL_ROW row;
char query_str[100];
int child_dead = 0;

int _GetHostName(char *buffer, int length);

unsigned char buffer[200];
unsigned char long_buffer[20];
unsigned char latt_buffer[20];
const char MESSAGE[] = "UBERSERVER*\n";
const int BACK_LOG = 5;

float lattitude;
float longitude;
float server_lattitude;
float server_longitude;
//---Används-för-att-hitta-rätt-poster-i-databas
float client_long;
float client_latt;
float server_long;
float server_latt;
//-----

/*
Namn: gps_convertToDecimal
Syfte: Gör om GPS koordinaten från DDMM.MMMM till DD.DDDD
*/
```

```

float gps_convertToDecimal(float gps_coord)
{
    float deg=0.0f;
    float decimal_part=0.0f;
    gps_coord = gps_coord/100;
    deg = floorf(gps_coord);
    decimal_part = gps_coord-deg;
    decimal_part *= 100;
    return (deg+(decimal_part/60));
}
/*
Namn: distance
Syfte: returnerar avståndet mellan två GPS koordinater
Algorithmen tar hänsyn till jordens krökning!
Returnerar: Avståndet i KM mellan två punkter
*/
float gps_distance(float lg1,float l1,float lg2,float l2)
{
    float d;
    float dg;
    float dl;
    float term1,term2,term3;
    lg1 = gps_convertToDecimal(lg1);
    l1 = gps_convertToDecimal(l1);
    lg2 = gps_convertToDecimal(lg2);
    l2 = gps_convertToDecimal(l2);

    dg = fabs(lg2-lg1);
    dl = fabs(l2-l1);
    term1 = 111.08956 * (dl + 0.000001);
    term2 = cos(l1 + (dl/2));
    term3 = (dg + 0.000001) / (dl + 0.000001);
    d = term1 / cos(atan(term2 * term3));
    // #ifndef SPHERODIAL
    // d = 1.852 * 60 * acos ( sin(l1) * sin(l2) + cos(l1) * cos(l2) *
cos(dg));
    // #endif
    return d;
}

int main(int argc, char *argv[])
{
    float AREASIZE=0.0f;
    float distance_between_points=0.0f;
    int pid_status=0;
    int pid_pid;
    unsigned int num_fields;
    int record_count;
    int cnt;
    unsigned char tst;

    int serverSocket = 0,
        on = 0,
        port = 0,
        status = 0,
        childPid = 0;
    struct hostent *hostPtr = NULL;
    char hostname[80] = "";
    struct sockaddr_in serverName = { 0 };

    if (2 != argc)

```

```
{
    fprintf(stderr, "Usage: %s <port>\n", argv[0]);
    exit(1);
}

port = atoi(argv[1]);
serverSocket = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
if (-1 == serverSocket)
{
    perror("socket()");
    exit(1);
}

on = 1;
status = setsockopt(serverSocket, SOL_SOCKET, SO_REUSEADDR,
    (const char *) &on, sizeof(on));
if (-1 == status)
{
    perror("setsockopt(...,SO_REUSEADDR,...)");
}
{
    struct linger linger = { 0 };
    linger.l_onoff = 1;
    linger.l_linger = 30;
    status = setsockopt(serverSocket, SOL_SOCKET, SO_LINGER,
        (const char *) &linger, sizeof(linger));

    if (-1 == status)
    {
        perror("setsockopt(...,SO_LINGER,...)");
    }
}

status = _GetHostName(hostname, sizeof(hostname));
if (-1 == status)
{
    perror("_GetHostName()");
    exit(1);
}
hostPtr = gethostbyname(hostname);
if (NULL == hostPtr)
{
    perror("gethostbyname()");
    exit(1);
}
(void) memset(&serverName, 0, sizeof(serverName));
(void) memcpy(&serverName.sin_addr, hostPtr->h_addr,
    hostPtr->h_length);

serverName.sin_family = AF_INET;
serverName.sin_port = htons(port); /* network-order */
status = bind(serverSocket, (struct sockaddr *) &serverName,
    sizeof(serverName));
if (-1 == status)
{
    perror("bind()");
    exit(1);
}
status = listen(serverSocket, BACK_LOG);
if (-1 == status)
{

```

```

        perror("listen()");
        exit(1);
    }
    for (;;)
    {
        struct sockaddr_in clientName = { 0 };
        int slaveSocket, clientLength = sizeof(clientName);

        (void) memset(&clientName, 0, sizeof(clientName));

        slaveSocket = accept(serverSocket,
            (struct sockaddr *) &clientName, &clientLength);
        if (-1 == slaveSocket)
        {
            perror("accept()");
            exit(1);
        }

        child_dead = 0;
        childPid = fork();
        switch (childPid)
        {
            case -1: /* Fel vid forkning */
                perror("fork()");
                exit(1);
            case 0: /* child process */
                close(serverSocket);
                if (-1 == getpeername(slaveSocket,
                    (struct sockaddr *) &clientName, &clientLength))
                {
                    perror("getpeername()");
                }
                else
                {
                    printf("Connection request from %s\n",
                        inet_ntoa(clientName.sin_addr));
                }
            // Läs vad Klienten skickar...
                memset(buffer, 0, sizeof(buffer)-1);
            // 1. läs av longitud från client
                read(slaveSocket, buffer, sizeof(buffer)-1);
                if (!(strcmp(buffer, "INFO")))
                {
                    // Okej skriv till client för att acca
                    write(slaveSocket, buffer, 1);
                    // läs av klientens svar
                    memset(buffer, 0, sizeof(buffer)-1);
                    read(slaveSocket, buffer, sizeof(buffer)-1);
                    connect_mysql();
                    //Ställ en fråga till SQL databasen
                    strcpy(query_str, "SELECT      sights      FROM      data      WHERE
location='");
                    strcat(query_str, buffer);
                    strcat(query_str, "'");
                    printf("%s\n", query_str);

                    mysql_query(&mysql, query_str);
                    result = mysql_use_result(&mysql);
                    row = mysql_fetch_row(result);

                    printf("%s\n", row[0]);

```

```

        read(slaveSocket,buffer,1);
        write(slaveSocket,row[0],strlen(row[0]));

        close(slaveSocket);
        mysql_close(&mysql);
        exit(0);
    }

    longitude = atof(buffer);
    memset(buffer,0,sizeof(buffer)-1);
    // 2. Svara att vi är klara
    write(slaveSocket,buffer,1);
    // 3. Läs av lattitud från client
    read(slaveSocket,buffer,sizeof(buffer)-1);
    lattitude = atof(buffer);

    // Läs in AREASIZE från klienten!
    //-----
    write(slaveSocket,buffer,1);
    read(slaveSocket,buffer,sizeof(buffer)-1);
    AREASIZE = atof(buffer);
    printf("Client says size of Area is:%f\n",AREASIZE);
    //-----
#ifdef trace_prints
    printf("Client longitude:%f\n",longitude);
    printf("Client lattitude:%f\n",lattitude);
#endif
    memset(buffer,0,sizeof(buffer)-1);

    //Skapa en anslutning mot mysql
    connect_mysql();
    strcpy(query_str,"SELECT * FROM data");

    mysql_query(&mysql,query_str);
    result = mysql_use_result(&mysql);

    while(row = mysql_fetch_row(result))
    {

        //Kolla upp longitud och lattitud från databasen och jämför
        //med den data som vi får från klienten
        strcpy(long_buffer,row[1]);
        server_longitude = atof(long_buffer);
        strcpy(long_buffer,row[2]);
        server_lattitude = atof(long_buffer);

        client_latt = lattitude;
        client_long = longitude;

        server_latt = server_lattitude;
        server_long = server_longitude;
        distance_between_points =
gps_distance(longitude,lattitude,server_longitude,server_lattitude);

        printf("Klient                                long:%f,Klient
latt:%f\n",longitude,lattitude);
        printf("Server                                long:%f,Server
latt:%f\n",server_longitude,server_lattitude);
        printf("Distance:%f \n",distance_between_points);

        // Okej Kolla upp om något i databasen är nära nog att visas!!

```

```
// AREASIZE fås från Klienten
    if (distance_between_points < AREASIZE)
    {
        write(slaveSocket, row[3], strlen(row[3]));
        read(slaveSocket, buffer, 1);
    }

    }
    mysql_close(&mysql);
    close(slaveSocket);
    exit(0);
default: /* Föräldern */
    close(slaveSocket);
    // Vänta tills barnen dött.. utan att dom blir zombies!
    pid_pid = wait(&pid_status);
}

}

while(child_dead == 0)
    kill(childPid);
return 0;
}

/*
Namn: connect_mysql
Syfte: Skapar en koppling mot en MySQL databas
*/
int connect_mysql(void)
{
    init_mysql();
    if (mysql_select_db(&mysql, MYSQL_DB)) {
        printf("Failed to select table: Error: %s\n", mysql_error(&mysql));
        mysql_close(&mysql);
        return 0;
    }

    return 1;
}

/*
Namn: _GetHostName
Syfte: Sätter rätt host namn
*/
int _GetHostName(char *buffer, int length)
{
    struct utsname sysname = { 0 };
    int status = 0;
    status = uname(&sysname);
    if (-1 != status)
    {
        strncpy(buffer, sysname.nodename, length);
    }
    return (status);
}

/*
Namn: init_mysql
Syfte: Initierar mysql databasen
*/
int init_mysql(void)
{
    mysql_init(&mysql);
```

```
    if
(!mysql_real_connect(&mysql,MYSQL_HOST,MYSQL_USERID,MYSQL_PASSWD,MYSQL_DB,2
4,NULL,0))
    {
        printf("Error while connecting to MySql:%s\n",mysql_error(&mysql));
        return 0;
    }
}
```

```
main.c

/*
  Turistinformationssystem
  Markus Lindblom , Johan Lindberg
*/

#include <stdio.h>
#include "../include/tnWidgets.h"
#include <sys/types.h>
#include <sys/socket.h>
#include <unistd.h>          /* read() */
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

//Globals
TN_WIDGET
*main_widget, *lbox1, *window1, *window2=NULL, *window0=NULL, *button1;
TN_WIDGET *button2, *textbox1, *label1, *label2, *mypicture;
TN_WIDGET *bluetoothpic, *window3=NULL, *window4=NULL, *win3button;
TN_WIDGET *listbox, *testtextbox, *win3infobox, *setupbutton, *arealabel;
TN_WIDGET *bluetooth_window=NULL, *setup_window;
TN_WIDGET *wait_window, *wait_label;
float AREASIZE=1.0f;
//--Client Vars-----
char server_ip[] = "81.172.140.78";
int remotePort = 1024;
struct hostent *hostPtr = NULL;
struct sockaddr_in serverName = { 0 };
char *remoteHost = NULL;
int clientSocket;
//-----
// För att spara undan info från servern
char *tst[256];
int free_counter=0;
// Buffer till client/server data
char buffer[256];

/*
GPS
*/
FILE *gps;
char gps_string[100];
char gps_buffer[100];
unsigned char lattitud[11];
unsigned char longitud[12];
/*
Namn: gps_open
Syfte: öppnar serieporten för att läsa ut gps positionen
*/
void gps_open(void)
{
    gps = fopen("/dev/ttySA1", "r");
}
/*
Namn: gps_close
Syfte: stänger serieporten
*/
void gps_close(void)
```

```
{
    fclose(gps);
}
/*
Namn: gps_iscorrect
Syfte: Kollar så att det är en valid GPS sträng
Returnerar: 0 vid ok 1 vid fel
*/
int gps_iscorrect(unsigned char *buffer)
{
    int result=0;
    // Kolla så att det är en valid Nema sträng
    result = strncmp("$GPGGA",buffer,6);
    if(!result)
        return 0; // Okej;
    else
        return 1; // Fel;
}
/*
Namn: gps_parse_to
Syfte: Används för att stega fram i den inlästa GPS strängen
*/
unsigned char* gps_parse_to(unsigned char *buffer,int cntr)
{
    unsigned char tst = 0;
    unsigned int i=0;
    for(i=0;i<cntr;i++)
    {
        while(strncmp(&tst,"",1))
        {
            tst = *buffer++;
        }
        tst = 0;
    }
    return buffer;
}

/*
Namn: gps_parse
Syfte: Plockar ut longitud & latitud ur en sträng
*/
int gps_parse(unsigned char *buffer)
{
    float latitude;
    int latt_degrees;
    float latt_minutes;
    float longitude;
    int long_degrees;
    float long_minutes;
    unsigned char *buffer_temp;
    // unsigned char lattitud[11];
    unsigned char lattitud_cardinal[3];
    // unsigned char longitud[12];
    unsigned char longitud_cardinal[3];
    unsigned char qualityfix[3];
    unsigned char numberofsat[3];
    unsigned char tst;
    unsigned int i = 0,x = 0;
    // Okej stega fram 2 komman
    buffer = gps_parse_to(buffer,2);
    while(strncmp(&tst,"",1))
```

```

    {
        lattitud[x] = *buffer;
        tst = *buffer++;
        x++;
    }
    x--; // remove komma
    lattitud[x]='\0';
// *buffer++;
    lattitud_cardinal[0] = *buffer;
    lattitud_cardinal[1] = '\0';
    *buffer++;
    *buffer++;
    tst = 0;
    x = 0;
    while(strncmp(&tst, ",", 1))
    {
        longitud[x] = *buffer;
        tst = *buffer++;
        x++;
    }
    x--;
    longitud[x] = '\0';
    longitud_cardinal[0] = *buffer;
    longitud_cardinal[1] = '\0';
    *buffer++;
    *buffer++;
    qualityfix[0] = *buffer;
    qualityfix[1] = '\0';
    *buffer++;
    *buffer++;
    numberofsat[0] = *buffer;
    *buffer++;
    numberofsat[1] = *buffer;
    numberofsat[2] = '\0';
}
/*
Namn: gps_getstring
Syfte: läser av en GPGLGA sträng från GPS enheten
*/
void gps_getstring(FILE *gps)
{
    memset(gps_buffer, 0, 99);
    while(gps_iscorrect(gps_buffer))
    {
        fflush(gps);
        memset(gps_buffer, 0, 99);
        fgets(&gps_buffer, 100, gps);
    }
}

/*
Namn: connect_to_server
Syfte: kopplar sig mot servern
*/
void connect_to_server(void)
{
    int status;
    clientSocket = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (-1 == clientSocket)
    {

```

```

        perror("socket()");
        exit(1);
    }
    hostPtr = gethostbyname(server_ip);
    if (NULL == hostPtr)
    {
        hostPtr = gethostbyaddr(remoteHost, strlen(remoteHost),
            AF_INET);
        if (NULL == hostPtr)
        {
            perror("Error resolving server address ");
            exit(1);
        }
    }
    serverName.sin_family = AF_INET;
    serverName.sin_port = htons(remotePort);
    (void) memcpy(&serverName.sin_addr, hostPtr->h_addr,
        hostPtr->h_length);
    status = connect(clientSocket, (struct sockaddr*) &serverName,
        sizeof(serverName));
    if (-1 == status)
    {
        perror("connect()");
        exit(1);
    }
}
/*
Namn: endapp
Syfte: avsluta applicationen
*/
void endapp(TN_WIDGET *widget, DATA_POINTER d)
{
    tnDestroyWidget(widget);
    tnEndApp();
}
/*
Namn: endthis
Syfte: avslutningsfunktion
*/
void endthis(TN_WIDGET *widget, DATA_POINTER d)
{
    tnDestroyWidget(window3);
    window3 = NULL;
    for (free_counter=0; free_counter<255; free_counter++)
    {
        if (tst[free_counter]!=0)
        {
            printf("Free:%d", free_counter);
            free(tst[free_counter]);
            tst[free_counter]=0;
        }
    }
    return;
}
/*
Namn: Generic Destroy
Syfte: Används för att avsluta widgets
*/
void generic_end(TN_WIDGET *widget, DATA_POINTER d)
{
    tnDestroyWidget(d);
}

```

```

    d = NULL;
    return;
}
/**
Generic waitbox
**/
void display_waitbox(int show, char *test)
{
    if (show)
    {
        // HÄR

        wait_window=tnCreateWidget(TN_WINDOW,main_widget,75,30,TN_HEIGHT,100,TN_WIDTH,180,TN_CAPTION,"Please wait...",TN_END);
        // wait_label =
        tnCreateWidget(TN_LABEL,wait_window,4,20,TN_CAPTION,"Contacting database please wait..",TN_END);
        wait_label =
        tnCreateWidget(TN_LABEL,wait_window,2,20,TN_CAPTION,test,TN_END);
    }
    else
    {
        tnDestroyWidget(wait_label);
        tnDestroyWidget(wait_window);
    }
}

/*
Namn: win3ok
Syfte: Här hämtas och visas informationen om den valda rubriken
*/
void win3ok(TN_WIDGET *widget, DATA_POINTER d)
{
    TN_WIDGET *win4button,*win4textbox;
    char **test;
    int **pos;
    int cnt;

    // Kontrollera så att användaren verkligen har valt något.
    tnGetSelectedListItems(listbox,&test,&cnt);
    if (cnt==--1)
        return;
    if (cnt==0)
        return;

    display_waitbox(1,"Downloading data from database!");
    tnSetText(testtextbox,test[0]);
    //-----
    connect_to_server();
    write(clientSocket,"INFO",4);
    //Vänta på svar från server
    read(clientSocket,buffer,1);
    // Skriv vad vi vill veta mer om till servern
    strcpy(buffer,test[0]);
    write(clientSocket,buffer,sizeof(buffer)-1);
    //Nolla buffert
    memset(buffer,0,sizeof(buffer)-1);
    //Läs in informationen om stället
    write(clientSocket,buffer,1);
    //-----

```

```

    read(clientSocket,buffer,sizeof(buffer)-1);
    printf("Directly after client read:%s\n",buffer);
    close(clientSocket);
//-----
    free(test[0]);
// Okej ta bort WAIT messet
    display_waitbox(0,"");

window4=tnCreateWidget(TN_WINDOW,main_widget,75,30,TN_HEIGHT,300,TN_WIDTH,3
00,TN_CAPTION,"Info",TN_END);
    win4textbox = tnCreateWidget(TN_TEXTBOX,window4,2,
2,TN_TEXTBOXTYPE,TN_MULTILINE,TN_DEFAULTTEXT,"",TN_WIDTH,
295,TN_HEIGHT,260,TN_END);
    win4button =
tnCreateWidget(TN_BUTTON,window4,8,265,TN_CAPTION,"CANCEL",TN_END);
    tnRegisterCallBack(win4button,CLICKED,generic_end,window4);

    tnGetSelectedListPos(listbox,&pos,&cnt);
    cnt = pos[0];
    tnSetText(win4textbox,buffer);
    return;
}

/*****
Namn: bluetooth_func
Syfte: Listar andra bluetooth enheter i närheten
*****/
void bluetooth_func(TN_WIDGET *button,DATA_POINTER ptr)
{
    char bt_devices[15][50];
    char *listbox_bt_devices[15];
    FILE *fd;
    int device_count=0;
    TN_WIDGET *bluetooth_ok_button,*bluetooth_listbox;

// Skapa ett nytt fönster
    bluetooth_window =
tnCreateWidget(TN_WINDOW,main_widget,0,0,TN_HEIGHT,440,TN_WIDTH,320,TN_CAPT
ION,"Devices in area",TN_END);
// Skapa OK knapp..
    bluetooth_ok_button =
tnCreateWidget(TN_BUTTON,bluetooth_window,20,400,TN_CAPTION,"OK",TN_END);
// Se till att användaren vet att vi fortfarande lever.
    display_waitbox(1,"Executing inquiry please wait..");
// Kör bting som anropar en modifierad version av bt_if.c
    system("bting -r");
// Öppna filen som alla bluetooth enheter har sparats till
    fd = fopen("result.txt","r");
    while(!(feof(fd)))
    {
        fgets(&bt_devices[device_count],50,fd);
        listbox_bt_devices[device_count] = &bt_devices[device_count];
        printf("%s\n",&bt_devices[device_count]);
        device_count++;
    }
    fclose(fd);
    display_waitbox(0,"");
    bluetooth_listbox =
tnCreateWidget(TN_LISTBOX,bluetooth_window,10,10,TN_WIDTH,240,TN_COUNT,devi
ce_count,TN_LISTITEMS,listbox_bt_devices,TN_END);

```



```

tnRegisterCallBack(blueetooth_ok_button,CLICKED,generic_end,bluetooth_window
);
}
//-----

/*
Här ska alltså alla sevärdigheternas rubriker listas i en listbox.
Maximala antalet sevärdheter per "ruta" är 20 stycken.
*/
void u_myfunc(TN_WIDGET *button,DATA_POINTER ptr)
{
    TN_WIDGET *win3okbutton,*win3label;
    int status;
    char area[15];
    char try[20][256];
    char *listbox_sights[20];
    int cnt=0;
    //Hm.. window3 BÖR vara noll här men kollar ändå för säkerhets skull
    if (window3!=NULL)
        return;
    // Nu måste GPRS kopplas upp annars går det åt skogen
    //-----
    //Koppla mot SQL servern
    // Här skall GPS läsas AV!!

    gps_open();
    gps_getstring(gps);
    printf("%s",gps_buffer);
    gps_parse(gps_buffer);
    printf("ÜBERTEST!\n");
    printf("Lat: %s\n",lattitud);
    printf("Lon: %s\n",longitud);
    gps_close();

    //Visa waitboxen
    display_waitbox(1,"Contacting database please wait..");
    connect_to_server();
    // Longitud & lattitud
    write(clientSocket,longitud,strlen(longitud));
    // write(clientSocket,"0.0",7);
    status = read(clientSocket,buffer,sizeof(buffer)-1);
    write(clientSocket,lattitud,strlen(lattitud));
    //write(clientSocket,"00.0",7);

    //-----
    // Okej skicka iväg hur stor AREA vi vill ha
    sprintf(area,"%f",AREASIZE);
    status = read(clientSocket,buffer,sizeof(buffer)-1);
    write(clientSocket,area,strlen(area));
    //-----

    memset(buffer,0,sizeof(buffer)-1);
    //-----
    // Nollställ tst
    for(cnt=0;cnt<255;cnt++)
        tst[cnt] = 0;
    //-----

```

```

    cnt=0;
// Om det ej fanns några sevärdheter så berätta det för användaren
    while (0 < (status = read(clientSocket,buffer,sizeof(buffer) - 1)))
    {
        printf("%s\n",buffer);
        sprintf(try[cnt],"%s",buffer);
        listBox_sights[cnt] = &try[cnt];
        // Säg till servern att vi vill ha mer data
        write(clientSocket,buffer,1);
        memset(buffer,0,sizeof(buffer)-1);
        cnt++;
    }
    close(clientSocket);
//Utifall att inga sevärdigheter hittades..
if (cnt==0)
{
    //          sprintf(try[0],"Nothing!");
    //          cnt++;
    cnt = -1;
}
//Ta bort boxen som säger åt användaren att vänta
display_waitbox(0,"");
if (cnt== -1)
{
    window3 =
tnCreateWidget(TN_WINDOW,main_widget,0,0,TN_HEIGHT,140,TN_WIDTH,220,TN_CAPT
ION,"List of sights",TN_END);    win3button =
tnCreateWidget(TN_BUTTON>window3,10,100,TN_CAPTION,"CANCEL",TN_END);
    listBox = tnCreateWidget(TN_LABEL>window3,10,10,TN_CAPTION,"Nothing of
interest in this area!",TN_FONTNAME,"impact",TN_FONTSIZE,15,TN_END);
}
else
{
    window3 =
tnCreateWidget(TN_WINDOW,main_widget,0,0,TN_HEIGHT,440,TN_WIDTH,420,TN_CAPT
ION,"List of sights",TN_END);
    win3button =
tnCreateWidget(TN_BUTTON>window3,20,400,TN_CAPTION,"CANCEL",TN_END);
    win3okbutton = tnCreateWidget(TN_BUTTON>window3,90,400,TN_CAPTION,"Get
info",TN_END);
    win3okbutton = tnCreateWidget(TN_BUTTON>window3,90,400,TN_CAPTION,"Get
info",TN_END);
    testtextbox = tnCreateWidget(TN_TEXTBOX>window3,150,20,TN_END);
    win3label =
tnCreateWidget(TN_LABEL>window3,150,5,TN_CAPTION,"Choice",TN_END);
    listBox =
tnCreateWidget(TN_LISTBOX>window3,10,20,TN_WIDTH,120,TN_COUNT,cnt,TN_LISTIT
EMS,listBox_sights,TN_END);
    tnRegisterCallBack(win3okbutton,CLICKED,win3ok,NULL);
}
    tnRegisterCallBack(win3button,CLICKED,endthis,NULL);
    return;
}
/*
Namn: setup_more
Syfte: Öka eller minska på AREASIZE beroende på om DATA_POINTER är NULL
eller ej
Kan öka&minska i två steg 0.1 eller 1.0
*/
void setup_more(TN_WIDGET *button,DATA_POINTER ptr)
{

```

```

    char area[15];
    generic_end(arealabel, NULL);
    if (ptr>10)
    {
        if (ptr==11)
            AREASIZE += 0.1;
        if (ptr==12)
            AREASIZE += 1.0;
    }
    else
    {
        //Området får ej bli mindre än 0.1
        if (ptr==1)
        {
            if (AREASIZE>0.1)
                AREASIZE -= 0.1;
        }
        if (ptr==2)
        {
            if (AREASIZE>1)
                AREASIZE -= 1.0;
        }
    }
    sprintf(area, "%.2f KM", AREASIZE);
    arealabel =
tnCreateWidget(TN_LABEL, setup_window, 70, 20, TN_CAPTION, area, TN_END);
    return;
}
/*
Namn: setup_func
Syfte: Detta fönster används för att ställa in storleken på det område som
användaren vill ha information om.
      (ökar och minskar globalen AREASIZE)
*/
void setup_func(TN_WIDGET *button, DATA_POINTER ptr)
{
    TN_WIDGET
*okbutton, *morebutton, *lessbutton, *muchmorebutton, *muchlessbutton;
    char area[15];
    sprintf(area, "%.2f KM", AREASIZE);
    setup_window =
tnCreateWidget(TN_WINDOW, main_widget, 20, 20, TN_HEIGHT, 200, TN_WIDTH, 200, TN_CA
PTION, "Setup", TN_END);
    okbutton =
tnCreateWidget(TN_BUTTON, setup_window, 20, 165, TN_CAPTION, "OK", TN_END);
    morebutton =
tnCreateWidget(TN_BUTTON, setup_window, 120, 80, TN_CAPTION, ">", TN_END);
    lessbutton =
tnCreateWidget(TN_BUTTON, setup_window, 20, 80, TN_CAPTION, "<", TN_END);
    muchmorebutton =
tnCreateWidget(TN_BUTTON, setup_window, 120, 120, TN_CAPTION, ">>", TN_END);
    muchlessbutton =
tnCreateWidget(TN_BUTTON, setup_window, 20, 120, TN_CAPTION, "<<", TN_END);
    arealabel =
tnCreateWidget(TN_LABEL, setup_window, 70, 20, TN_CAPTION, area, TN_END);
    tnRegisterCallBack(morebutton, CLICKED, setup_more, 11);
    tnRegisterCallBack(muchmorebutton, CLICKED, setup_more, 12);
    tnRegisterCallBack(lessbutton, CLICKED, setup_more, 1);
    tnRegisterCallBack(muchlessbutton, CLICKED, setup_more, 2);
    tnRegisterCallBack(okbutton, CLICKED, generic_end, setup_window);
    return;
}

```

```
}
/*
Namn: main
Syfte: Första fönstret i GUI
*/
int main(int argc, char **argv)
{
    TN_WIDGET *bluetooth_label, *setup_label;
    main_widget=tnAppInitialize(argc, argv);
    //Första fönstret
    window1=tnCreateWidget(TN_WINDOW, main_widget, 0, 0, TN_HEIGHT, 240, TN_WIDTH, 320, TN_CAPTION, "Portable information system!", TN_END);

    mypicture=tnCreateWidget(TN_BUTTON, window1, 20, 20, TN_HEIGHT, 45, TN_WIDTH, 45,
        TN_PIXMAP, TN_TRUE,
        TN_FILENAME, "/root/mosaic.gif",
        TN_END);
    bluetoothpic=tnCreateWidget(TN_BUTTON, window1, 20, 70, TN_HEIGHT, 45,
        TN_WIDTH, 45,
        TN_PIXMAP, TN_TRUE,
        TN_FILENAME, "/root/bluetooth.gif",
        TN_END);

    setupbutton=tnCreateWidget(TN_BUTTON, window1, 20, 120, TN_HEIGHT, 45,
        TN_WIDTH, 45, TN_PIXMAP, TN_FALSE, TN_CAPTION, "Setup", TN_END);
    labell=tnCreateWidget(TN_LABEL, window1, 70, 30, TN_CAPTION, "Check
area for sights", TN_FONTNAME, "impact", TN_FONTSIZE, 15, TN_END);
    bluetooth_label =
tnCreateWidget(TN_LABEL, window1, 70, 80, TN_CAPTION, "Check area for other
devices", TN_FONTNAME, "impact", TN_FONTSIZE, 15, TN_END);
    setup_label =
tnCreateWidget(TN_LABEL, window1, 70, 130, TN_CAPTION, "Change
configuration", TN_FONTNAME, "impact", TN_FONTSIZE, 15, TN_END);
    tnRegisterCallBack(window1, CLOSED, endapp, NULL);
    tnRegisterCallBack(mypicture, CLICKED, u_myfunc, NULL);
    tnRegisterCallBack(bluetoothpic, CLICKED, bluetooth_func, NULL);
    tnRegisterCallBack(setupbutton, CLICKED, setup_func, NULL);
    tnMainLoop();
    return 0;
}
```